



INTERNATIONAL CYBERSECURITY & DIGITAL FORENSIC ACADEMY
(ICDFA)

INT301 Week 1 Lab

Mastering the Linux Command Line for Ethical Hacking

Ethical Hacking and Digital Forensics Internship
Phase 1 - Week 1

Name: Olatunji Ramon Balogun

Student ID: 2025/INT/12584

INSTRUCTOR NAME: Mr. Aminu Idris AMCPA

2025/INT/12584

12 December 2025

Introduction

Welcome to the first week of your intensive training in cybersecurity and ethical hacking. This lab is designed to provide you with a comprehensive, hands-on introduction to the Linux command line, a fundamental tool for any cybersecurity professional.

You will start by exploring the Kali Linux environment, then move on to advanced file and directory operations, permission management, and essential text processing techniques. By the end of this lab, you will have a solid foundation for the more advanced topics to be covered in the coming weeks.

This lab combines several key areas into a single, unified learning experience. You will progress from basic navigation to complex command-line wizardry, all within the context of practical, real-world scenarios that you will encounter as a security professional.

Executive Summary

This lab built basic abilities needed to work well in Kali Linux, a go to setup for ethical hackers and digital investigators. Main attention went to getting comfortable with terminal commands the core tool in security tasks. Learners checked out the desktop layout, ran key command-line actions, also handled complex file and folder jobs. They tried editing file details, changing access rights plus user control, while using lookup and text tools to study info. Other tasks involved packing files into archives, shrinking their size, or watching running programs and storage space.

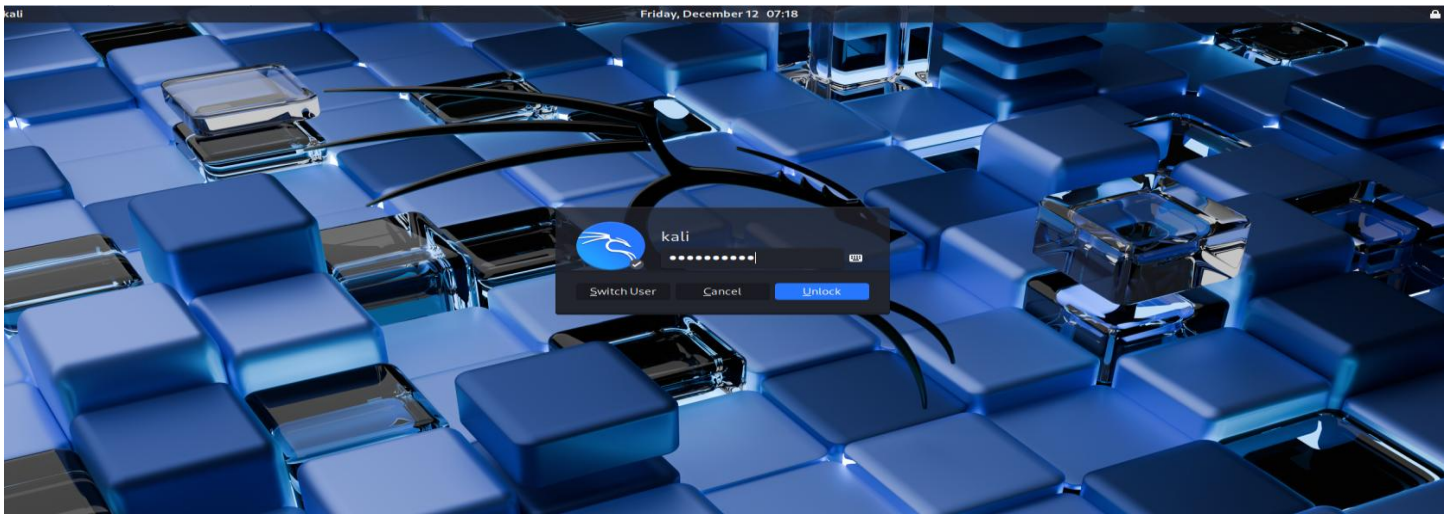
The results from this lab showed how well learners can move around and control Linux environments, highlighting why knowing terminal commands really matters when doing actual security tests or digital forensics. Since finishing these activities, trainees now have a solid base skill set that'll help tackle tougher cyber challenges coming up later. What made this session stand out wasn't just performing tasks correctly instead, it focused on thinking critically about findings while connecting them to real job situations.

Part 1: The Kali Linux Environment

Kali Linux is a Debian-derived Linux distribution designed for digital forensics and penetration testing. It comes pre-installed with a vast collection of security tools, making it the top choice for security professionals. In this section, you will get comfortable with the Kali Linux Graphical User Interface (GUI) and the powerful command-line shell.

1.1: Exploring the GUI

1. Familiarize yourself with the desktop environment. Explore the applications menu, the file manager, and the system settings.

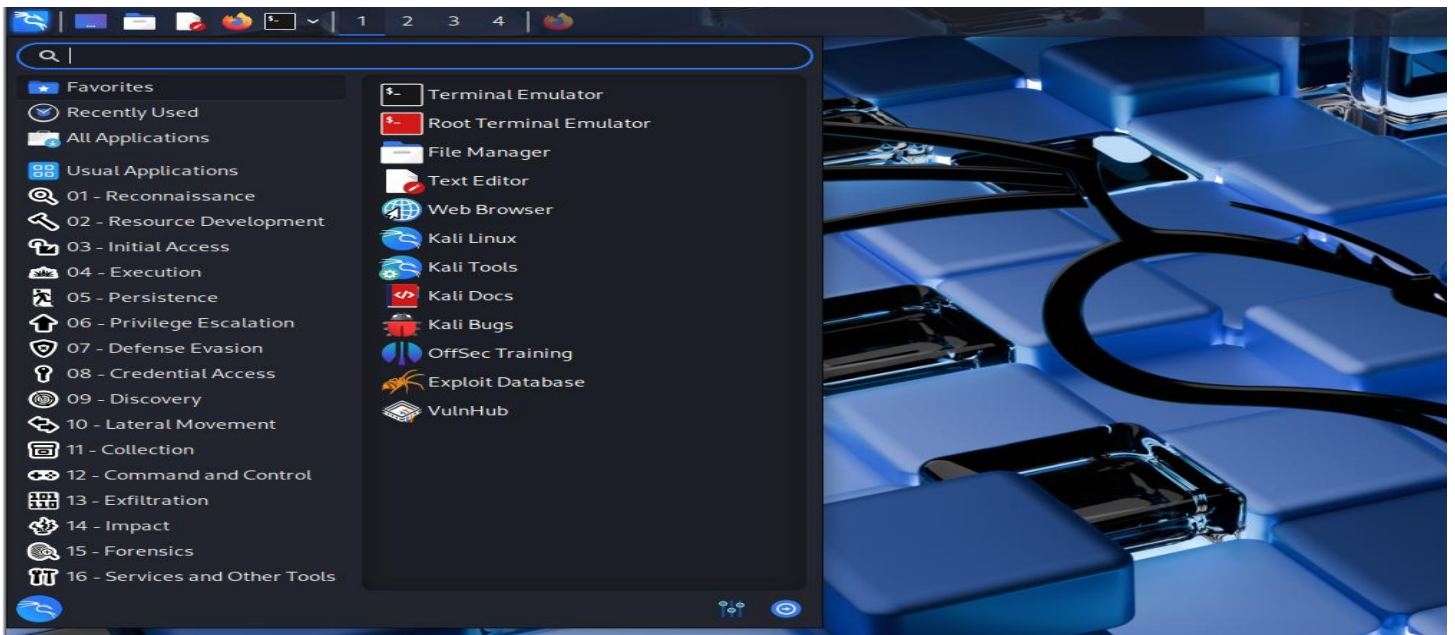


Login page of Kali Linux Graphical User Interface (GUI)

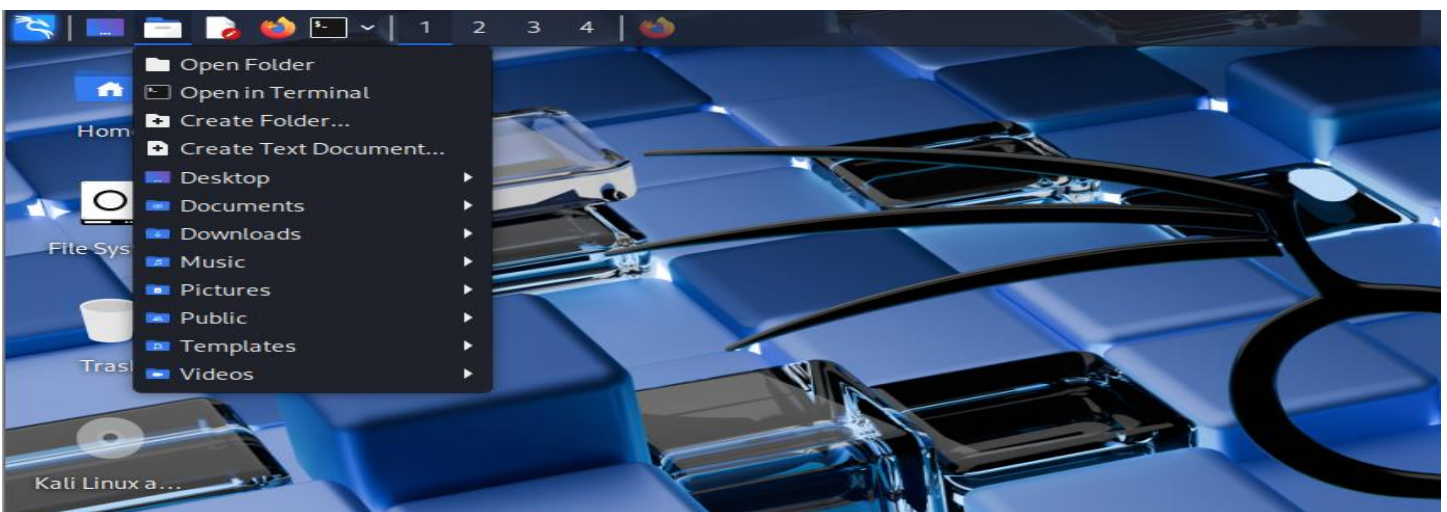


Full desktop environment of the Kali Linux Graphical User Interface (GUI)





Picture of the Application menu: It makes finding built in tools easier, since they help with checking networks, breaking passwords, going through logs, looking into malware, also keeping evidence safe



file manager: provides a graphical interface for browsing, organizing, and managing files and directories...

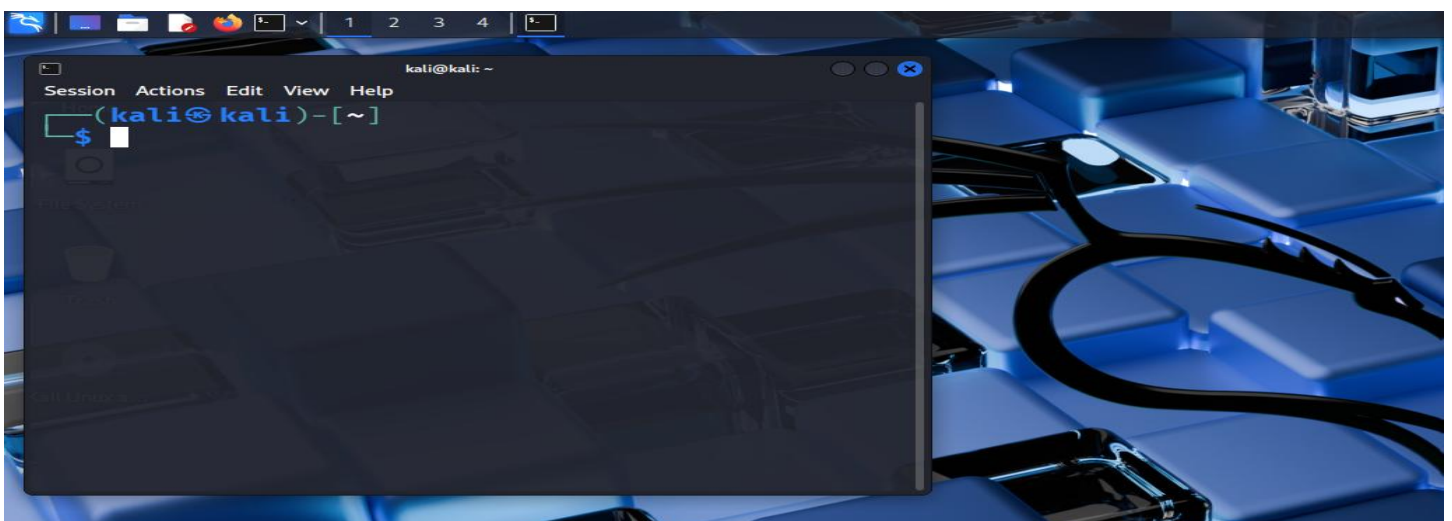


Picture of the web browser



Picture of the web browser: The in-built browser that comes with kali linux is a Mozilla fire fox for Web access, testing web apps and using web based tools/documentation

2. Open a terminal window. The terminal is your primary tool for interacting with the system. You can find it in the applications menu or by using a keyboard shortcut (usually Ctrl+Alt+T).



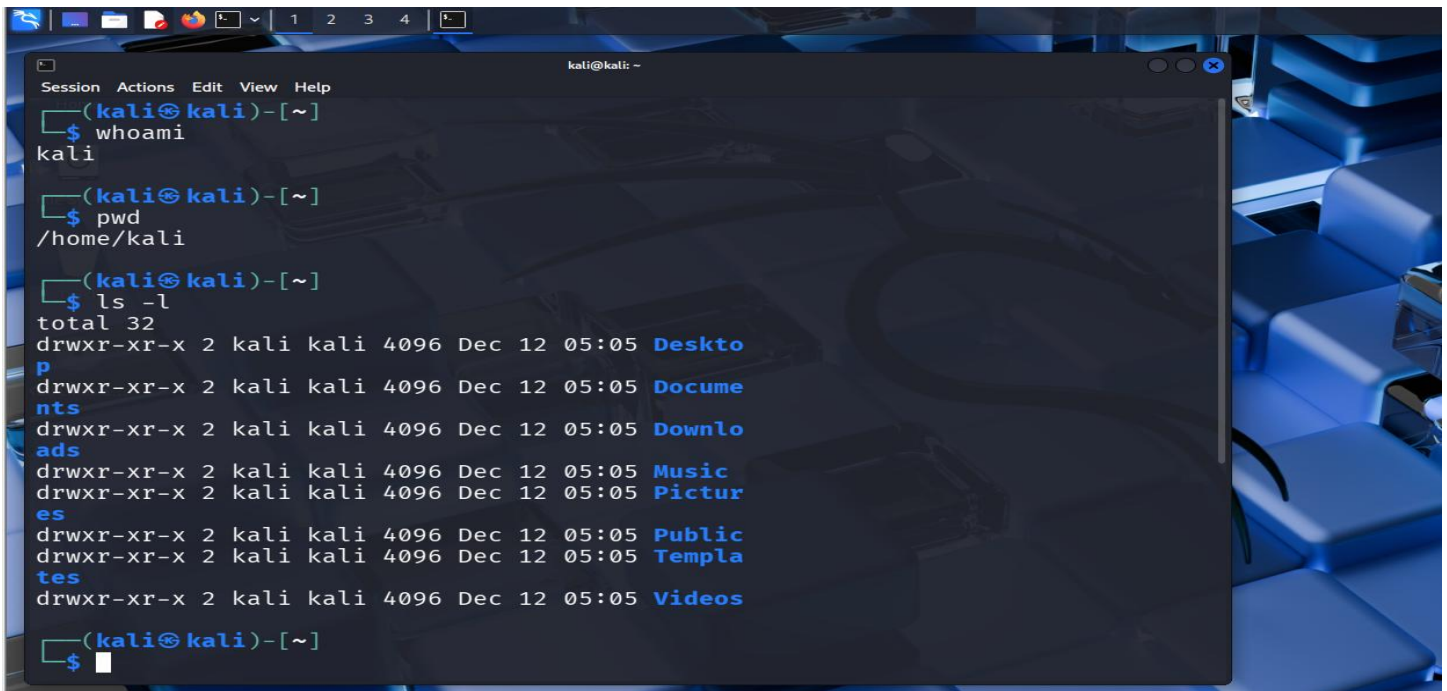
1.2: Mastering the Shell

The shell is a command-line interpreter that allows you to run commands and scripts. Mastering the shell is crucial for efficiency and automation.

- Exercise: Open a terminal and type the following commands, pressing Enter after each one:

```
whoami  
pwd  
ls -l
```

- Reflect on the output of each command. What information does each command provide?
 - whoami: Shows your current user.
 - pwd: Displays the current working directory.
 - ls -l: Lists the files and directories in the current directory in long format, showing permissions, ownership, size, and modification date.



```
kali@kali: ~  
$ whoami  
kali  
$ pwd  
/home/kali  
$ ls -l  
total 32  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Desktop  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Documents  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Downloads  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Music  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Pictures  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Public  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Templates  
drwxr-xr-x 2 kali kali 4096 Dec 12 05:05 Videos  
$
```

Part 2: Advanced Directory and File Operations

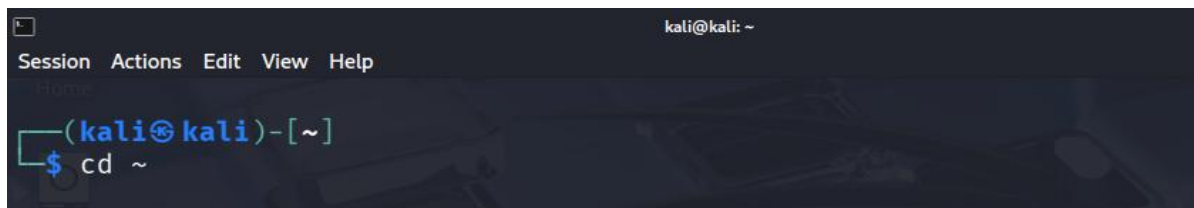
Now that you are familiar with the basic shell commands, it's time to dive into more advanced operations for managing files and directories. These skills are essential for organizing your work during a penetration test.

2.1: Advanced Directory Navigation & Creation

Efficiently creating and navigating complex directory structures is a key skill.

3. Navigate to your home directory:

```
cd ~
```

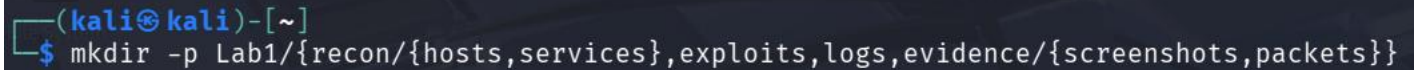


```
kali@kali: ~  
Session Actions Edit View Help  
Home  
(kali@kali)-[~]  
$ cd ~
```

Cd ~ : change directory

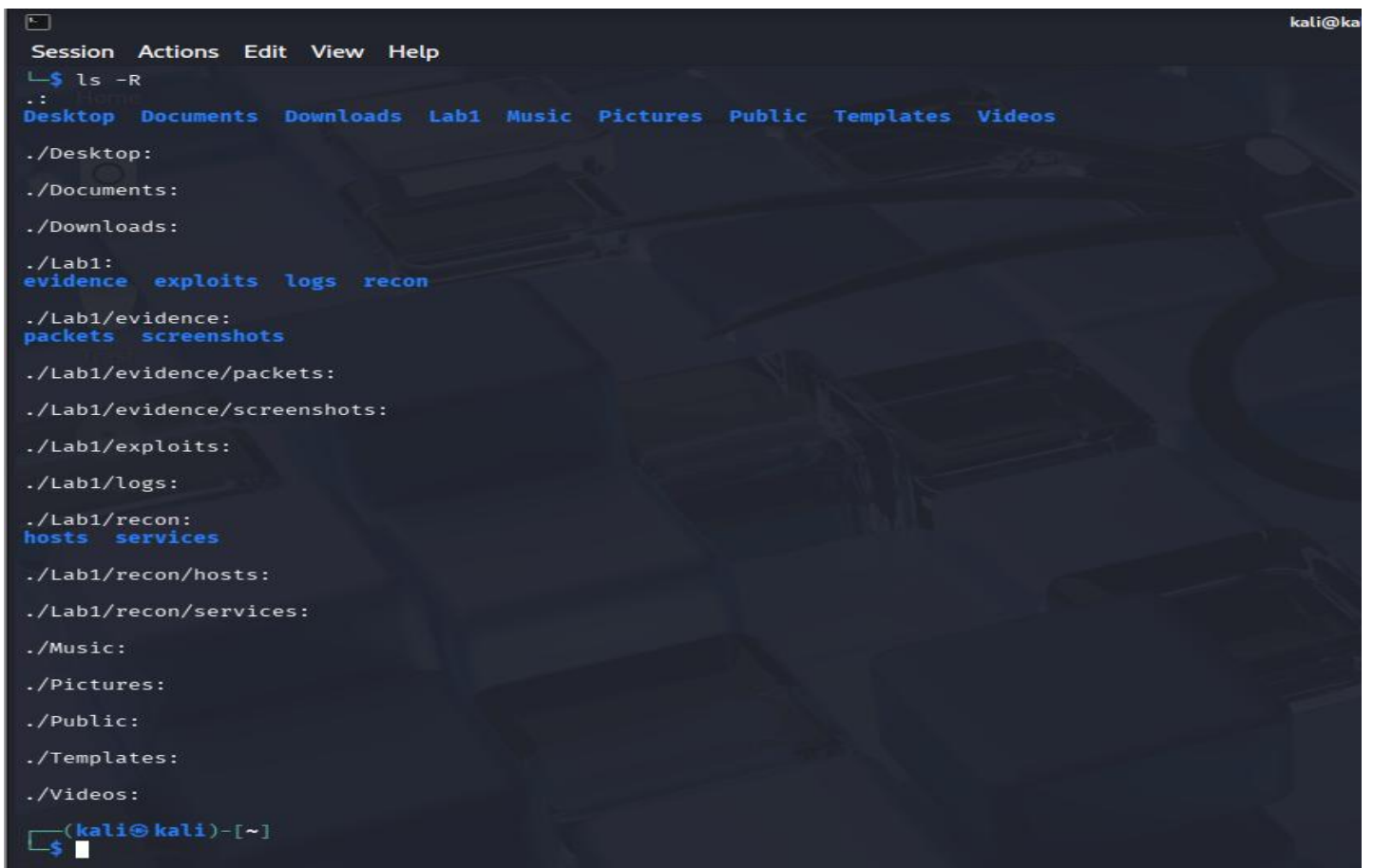
4. Create a complex directory structure in one command:

```
mkdir -p  
Lab1/{recon/{hosts,services},exploits,logs,evidence/{screenshots,pack  
ets}}
```



```
(kali@kali)-[~]  
$ mkdir -p Lab1/{recon/{hosts,services},exploits,logs,evidence/{screenshots,packets}}
```

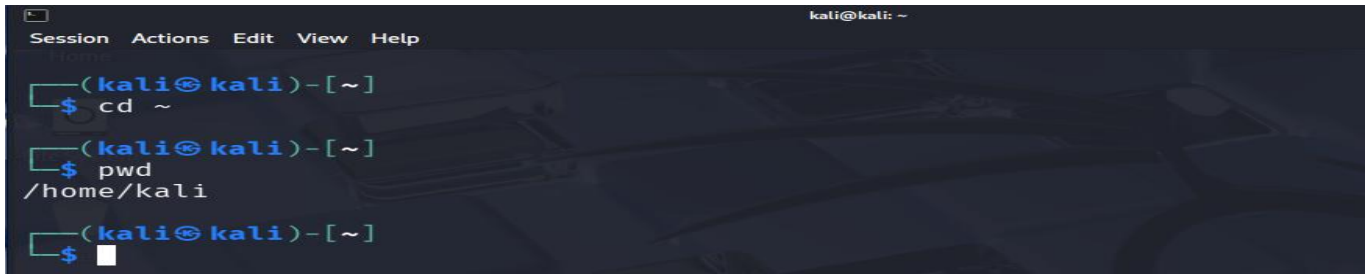
5. Use tree or ls -R to view the recursive structure you just created.



```
kali@ka  
Session Actions Edit View Help  
$ ls -R  
.:  
Desktop Documents Downloads Lab1 Music Pictures Public Templates Videos  
./Desktop:  
./Documents:  
./Downloads:  
./Lab1:  
evidence exploits logs recon  
./Lab1/evidence:  
packets screenshots  
./Lab1/evidence/packets:  
./Lab1/evidence/screenshots:  
./Lab1/exploits:  
./Lab1/logs:  
./Lab1/recon:  
hosts services  
./Lab1/recon/hosts:  
./Lab1/recon/services:  
./Music:  
./Pictures:  
./Public:  
./Templates:  
./Videos:  
(kali@kali)-[~]  
$
```

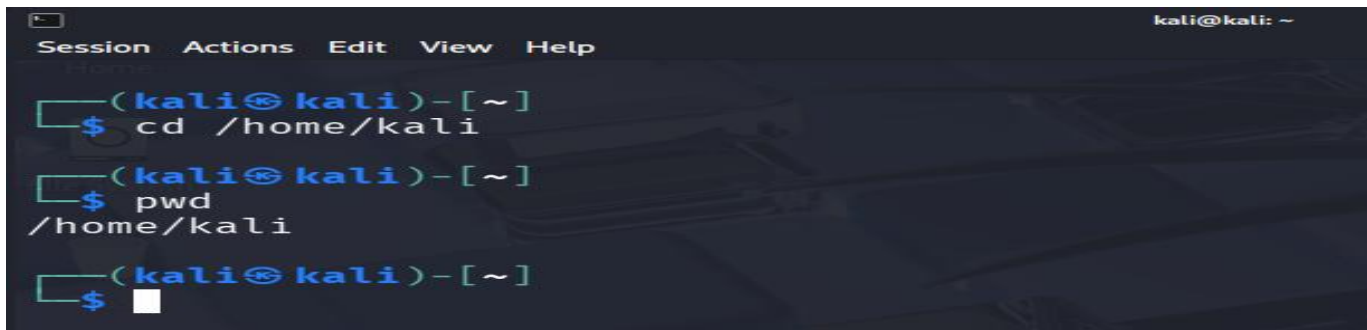
Exercises

- What is the difference between `cd ~` and `cd /home/kali`?



```
kali@kali: ~  
Session Actions Edit View Help  
Home  
(kali@kali)-[~]  
$ cd ~  
(kali@kali)-[~]  
$ pwd  
/home/kali  
(kali@kali)-[~]  
$
```

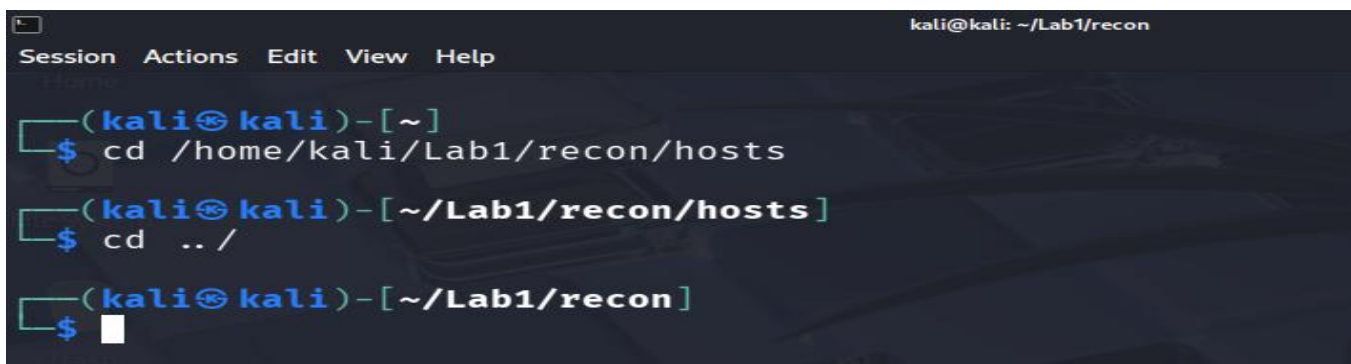
The `cd ~` - it's a quick way to jump into the current user's home folder. So, if you're using Kali, typing `cd ~` lands you in `/home/kali`.



```
kali@kali: ~  
Session Actions Edit View Help  
Home  
(kali@kali)-[~]  
$ cd /home/kali  
(kali@kali)-[~]  
$ pwd  
/home/kali  
(kali@kali)-[~]  
$
```

Now, `cd /home/kali` is different that's what we call a full path. It goes straight to the kali user's directory no matter who's logged in. Unlike shortcuts, this one doesn't change based on the user

- Explain how the `../` syntax works in directory navigation.
`../` means go back to the folder above where you are now. Using `cd ../` shifts you one step up in the file structure., You can stack them to jump several steps- like `cd ../` to climb higher fast. Say you are inside `/home/kali/Labs/recon/hosts`. Run `cd ../` from there you will land in previous folders i.e, go up one folder



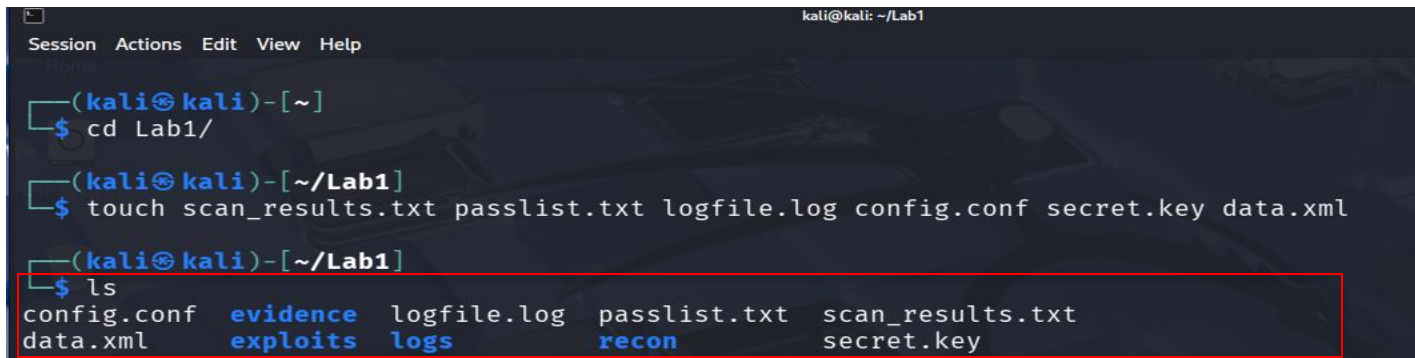
```
kali@kali: ~/Lab1/recon  
Session Actions Edit View Help  
Home  
(kali@kali)-[~]  
$ cd /home/kali/Lab1/recon/hosts  
(kali@kali)-[~/Lab1/recon/hosts]  
$ cd ../  
(kali@kali)-[~/Lab1/recon]  
$
```


2.2: Comprehensive File Operations

In this section, you'll practice creating, copying, moving, and deleting files.

6. Create multiple practice files:

```
cd ~/Lab1
touch scan_results.txt passlist.txt logfile.log config.conf
secret.key data.xml
```



A terminal window titled 'kali@kali: ~/Lab1' showing the execution of commands to create files and list the directory. The files created are scan_results.txt, passlist.txt, logfile.log, config.conf, secret.key, and data.xml. The 'ls' command output shows these files with color-coded categories: evidence (scan_results.txt, secret.key), exploits (config.conf, data.xml), logs (logfile.log), recon (passlist.txt), and a file named 'recon'.

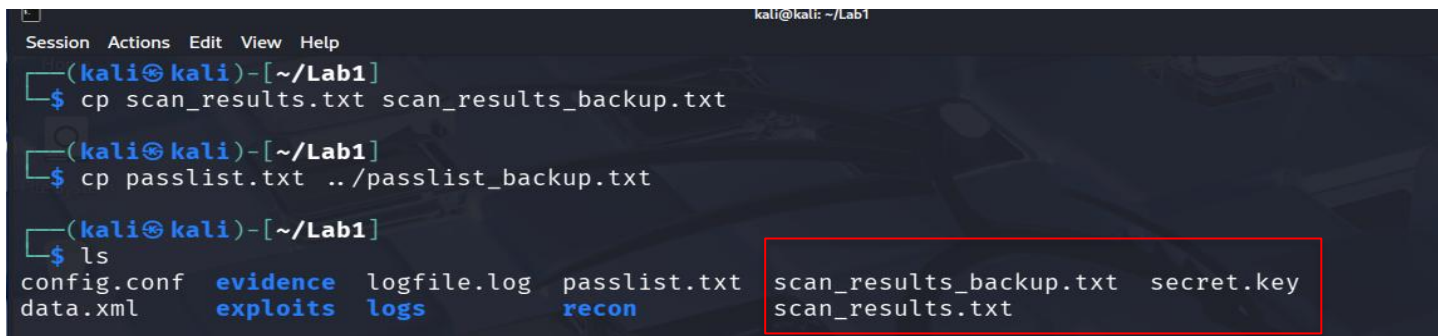
```
(kali@kali)-[~]
$ cd Lab1/

(kali@kali)-[~/Lab1]
$ touch scan_results.txt passlist.txt logfile.log config.conf secret.key data.xml

(kali@kali)-[~/Lab1]
$ ls
config.conf  evidence  logfile.log  passlist.txt  scan_results.txt
data.xml     exploits  logs         recon         secret.key
```

7. Copy files with verification:

```
cp scan_results.txt scan_results_backup.txt
cp passlist.txt ../passlist_backup.txt
```



A terminal window titled 'kali@kali: ~/Lab1' showing the execution of copy commands and a subsequent 'ls' command to verify the files. The files copied are scan_results.txt to scan_results_backup.txt and passlist.txt to ../passlist_backup.txt. The 'ls' command output shows the updated directory contents, including the new backup files.

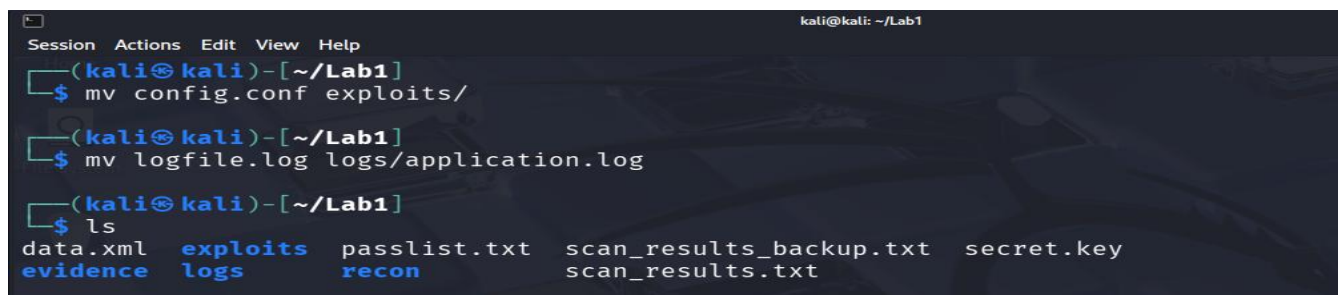
```
(kali@kali)-[~/Lab1]
$ cp scan_results.txt scan_results_backup.txt

(kali@kali)-[~/Lab1]
$ cp passlist.txt ../passlist_backup.txt

(kali@kali)-[~/Lab1]
$ ls
config.conf  evidence  logfile.log  passlist.txt  scan_results_backup.txt  secret.key
data.xml     exploits  logs         recon         scan_results.txt
```

8. Move and rename files:

```
mv config.conf exploits/
mv logfile.log logs/application.log
```



A terminal window titled 'kali@kali: ~/Lab1' showing the execution of move and rename commands and a subsequent 'ls' command to verify the changes. The files moved are config.conf to exploits/ and logfile.log to logs/application.log. The 'ls' command output shows the updated directory contents, including the moved files.

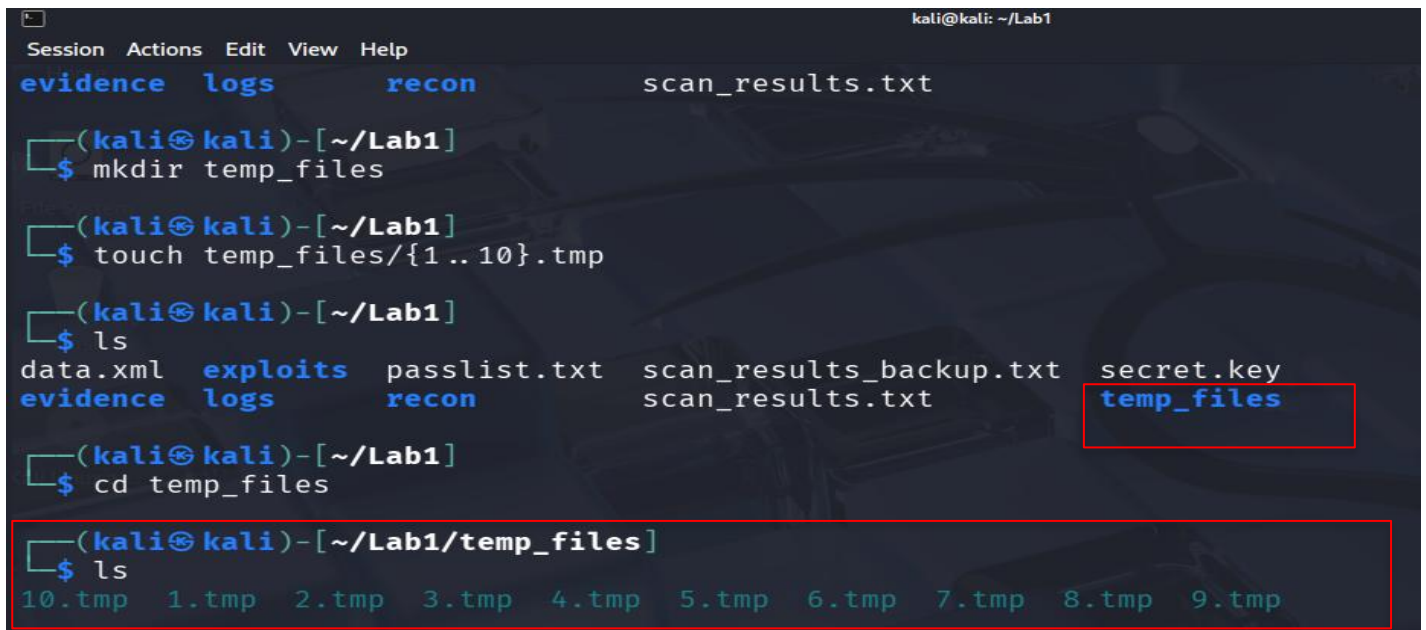
```
(kali@kali)-[~/Lab1]
$ mv config.conf exploits/

(kali@kali)-[~/Lab1]
$ mv logfile.log logs/application.log

(kali@kali)-[~/Lab1]
$ ls
data.xml  exploits  passlist.txt  scan_results_backup.txt  secret.key
evidence  logs      recon         scan_results.txt
```

9. Bulk delete files:

```
mkdir temp_files
touch temp_files/{1..10}.tmp
rm temp_files/*.tmp
```



A terminal window titled 'kali@kali: ~/Lab1' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the following commands and output:

```
(kali@kali)-[~/Lab1]
$ mkdir temp_files

(kali@kali)-[~/Lab1]
$ touch temp_files/{1..10}.tmp

(kali@kali)-[~/Lab1]
$ ls
data.xml  exploits  passlist.txt  scan_results_backup.txt  secret.key
evidence  logs      recon         scan_results.txt         temp_files
```

The 'temp_files' directory is highlighted with a red box. The next command is:

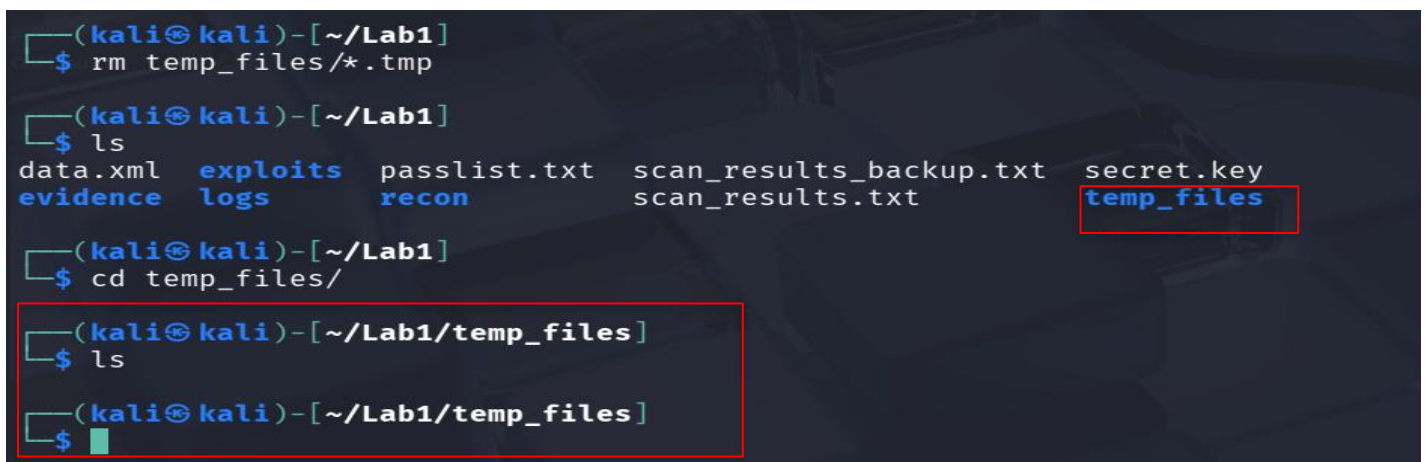
```
(kali@kali)-[~/Lab1]
$ cd temp_files

(kali@kali)-[~/Lab1/temp_files]
$ ls
10.tmp  1.tmp  2.tmp  3.tmp  4.tmp  5.tmp  6.tmp  7.tmp  8.tmp  9.tmp
```

This second 'ls' command and its output are also highlighted with a red box.

Exercises

- Verify the copy, move, and delete operations using the ls command.



A terminal window titled 'kali@kali: ~/Lab1' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows the following commands and output:

```
(kali@kali)-[~/Lab1]
$ rm temp_files/*.tmp

(kali@kali)-[~/Lab1]
$ ls
data.xml  exploits  passlist.txt  scan_results_backup.txt  secret.key
evidence  logs      recon         scan_results.txt         temp_files
```

The 'temp_files' directory is highlighted with a red box. The next command is:

```
(kali@kali)-[~/Lab1]
$ cd temp_files/

(kali@kali)-[~/Lab1/temp_files]
$ ls
```

This 'ls' command and its output are highlighted with a red box. The final command shown is:

```
(kali@kali)-[~/Lab1/temp_files]
$
```

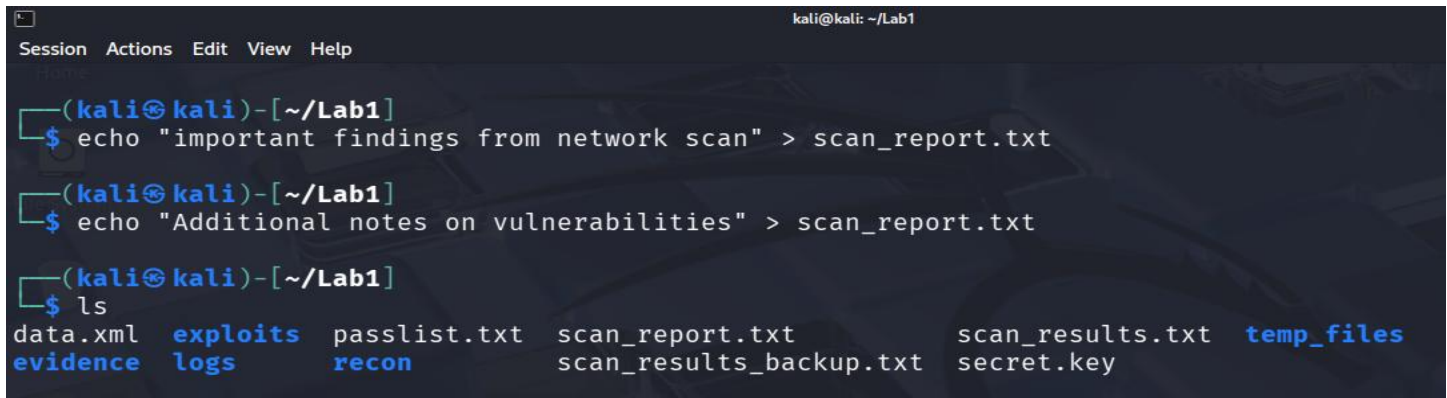
Part 3: File Content Management

Managing the content of files is just as important as managing the files themselves. This section covers creating, viewing, and searching within files.

3.1: Creating and Viewing File Content

10. Create a file and add content to it:

```
echo "Important findings from network scan" > scan_report.txt
echo "Additional notes on vulnerabilities" >> scan_report.txt
```

A terminal window titled 'kali@kali: ~/Lab1' with a menu bar (Session, Actions, Edit, View, Help). It shows three commands: 1. 'echo "important findings from network scan" > scan_report.txt', 2. 'echo "Additional notes on vulnerabilities" > scan_report.txt', and 3. 'ls'. The 'ls' command output lists files: data.xml, exploits, passlist.txt, scan_report.txt, scan_results.txt, temp_files, evidence, logs, recon, scan_results_backup.txt, and secret.key.

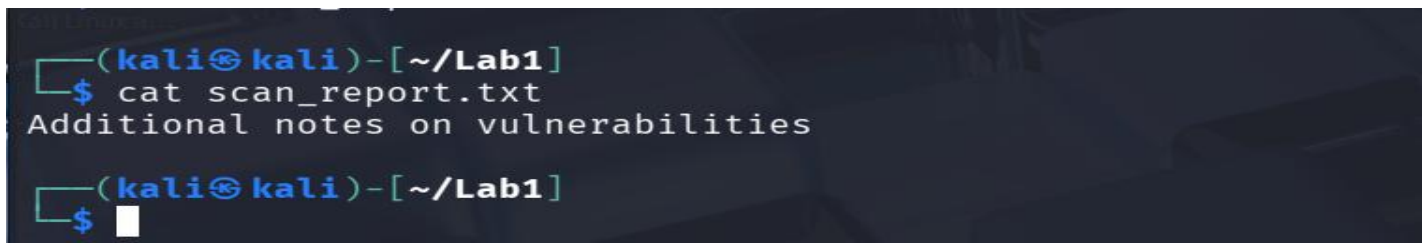
```
(kali@kali)-[~/Lab1]
$ echo "important findings from network scan" > scan_report.txt

(kali@kali)-[~/Lab1]
$ echo "Additional notes on vulnerabilities" > scan_report.txt

(kali@kali)-[~/Lab1]
$ ls
data.xml  exploits  passlist.txt  scan_report.txt  scan_results.txt  temp_files
evidence  logs      recon         scan_results_backup.txt  secret.key
```

11. View the content of the file:

```
cat scan_report.txt
```

A terminal window titled 'kali@kali: ~/Lab1' showing the command 'cat scan_report.txt' and its output: 'Additional notes on vulnerabilities'.

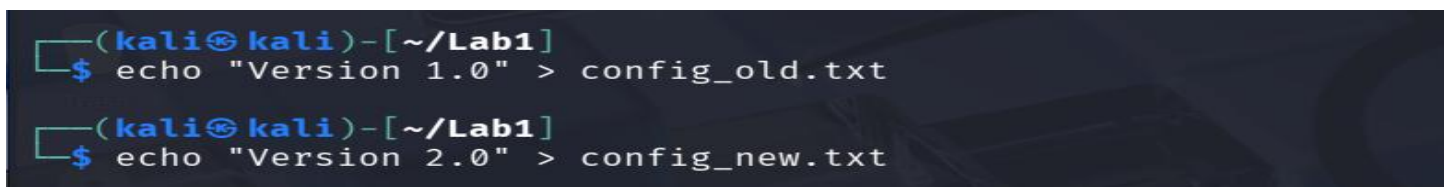
```
(kali@kali)-[~/Lab1]
$ cat scan_report.txt
Additional notes on vulnerabilities

(kali@kali)-[~/Lab1]
$
```

3.2: Comparing File Differences

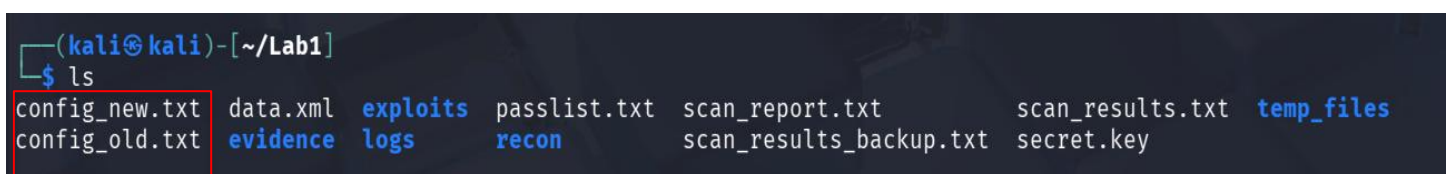
12. Create two slightly different files:

```
echo "Version 1.0" > config_old.txt
echo "Version 2.0" > config_new.txt
```

A terminal window titled 'kali@kali: ~/Lab1' showing two commands: 'echo "Version 1.0" > config_old.txt' and 'echo "Version 2.0" > config_new.txt'.

```
(kali@kali)-[~/Lab1]
$ echo "Version 1.0" > config_old.txt

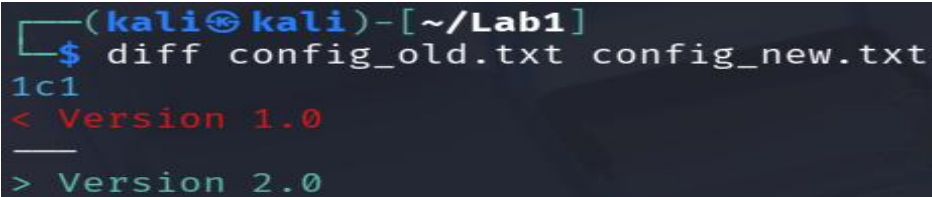
(kali@kali)-[~/Lab1]
$ echo "Version 2.0" > config_new.txt
```

A terminal window titled 'kali@kali: ~/Lab1' showing the command 'ls'. The output lists files, with 'config_new.txt' and 'config_old.txt' highlighted by a red box.

```
(kali@kali)-[~/Lab1]
$ ls
config_new.txt  data.xml  exploits  passlist.txt  scan_report.txt  scan_results.txt  temp_files
config_old.txt  evidence  logs      recon         scan_results_backup.txt  secret.key
```


13. Compare the files:

```
diff config_old.txt config_new.txt
```



```
(kali㉿kali)-[~/Lab1]
$ diff config_old.txt config_new.txt
1c1
< Version 1.0
---
> Version 2.0
```

Exercise

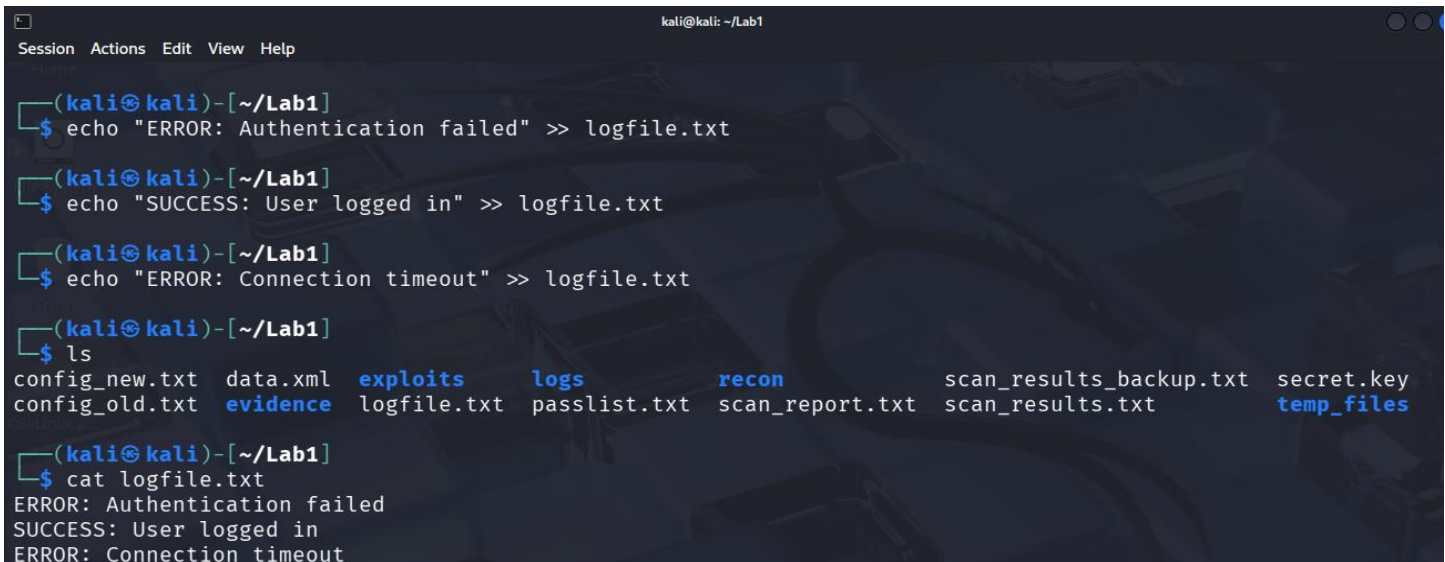
- What does the output of diff show you?

The output of diff show that the version 1 was updated in the new file which Version 2.

3.3: Searching Within Files

14. Create a log file with some sample data:

```
echo "ERROR: Authentication failed" >> logfile.txt
echo "SUCCESS: User logged in" >> logfile.txt
echo "ERROR: Connection timeout" >> logf
```



```
kali@kali: ~/Lab1
Session Actions Edit View Help

(kali㉿kali)-[~/Lab1]
$ echo "ERROR: Authentication failed" >> logfile.txt

(kali㉿kali)-[~/Lab1]
$ echo "SUCCESS: User logged in" >> logfile.txt

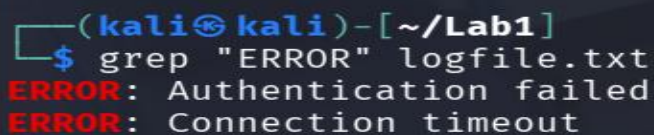
(kali㉿kali)-[~/Lab1]
$ echo "ERROR: Connection timeout" >> logfile.txt

(kali㉿kali)-[~/Lab1]
$ ls
config_new.txt  data.xml  exploits  logs      recon      scan_results_backup.txt  secret.key
config_old.txt  evidence  logfile.txt  passlist.txt  scan_report.txt  scan_results.txt         temp_files

(kali㉿kali)-[~/Lab1]
$ cat logfile.txt
ERROR: Authentication failed
SUCCESS: User logged in
ERROR: Connection timeout
```

15. Search for specific patterns in the file:

```
grep "ERROR" logfile.txt
```



```
(kali㉿kali)-[~/Lab1]
$ grep "ERROR" logfile.txt
ERROR: Authentication failed
ERROR: Connection timeout
```

Exercise

- Use `grep -c` to count how many ERROR messages are in the file.

```
(kali㉿kali)-[~/Lab1]
$ grep -c "ERROR" logfile.txt
2
```

3.4: Viewing Partial File Content

16. View the beginning and end of a file:

```
head -3 logfile.txt
tail -2 logfile.txt
```

Exercise

- Create a command that shows only lines 2-3 of logfile.txt using head and tail.

```
(kali㉿kali)-[~/Lab1]
$ head -3 logfile.txt
ERROR: Authentication failed
SUCCESS: User logged in
ERROR: Connection timeout
```

```
head -3 logfile.txt
```

```
(kali㉿kali)-[~/Lab1]
$ head -2 logfile.txt
ERROR: Authentication failed
SUCCESS: User logged in
```

```
tail -2 logfile.txt
```

Part 4: Advanced Permission Exercises

Understanding and managing file permissions is a cornerstone of Linux security. This section will provide you with hands-on experience in manipulating file permissions and ownership.

4.1: Checking and Modifying Permissions

Exercises

- After each `chmod` command, run `ls -l` and explain what changed.
- What do each of these octal numbers (755, 600, 644) represent in terms of permissions?

17. Create some test files:

```
cd ~/Lab1
touch secret_file.txt admin_tool.sh public_read.txt
```

```
kali@kali: ~/Lab1
Session Actions Edit View Help

(kali@kali)-[~/Lab1]
$ touch secret_file.txt admin_tool.sh public_read.txt

(kali@kali)-[~/Lab1]
$ ls -l
total 36
-rw-rw-r-- 1 kali kali 0 Dec 12 12:39 admin_tool.sh
-rw-rw-r-- 1 kali kali 12 Dec 12 11:48 config_new.txt
-rw-rw-r-- 1 kali kali 12 Dec 12 11:45 config_old.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 data.xml
drwxrwxr-x 4 kali kali 4096 Dec 12 09:51 evidence
drwxrwxr-x 2 kali kali 4096 Dec 12 11:06 exploits
-rw-rw-r-- 1 kali kali 79 Dec 12 12:10 logfile.txt
drwxrwxr-x 2 kali kali 4096 Dec 12 11:07 logs
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 passlist.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 12:39 public_read.txt
drwxrwxr-x 4 kali kali 4096 Dec 12 09:51 recon
-rw-rw-r-- 1 kali kali 36 Dec 12 11:38 scan_report.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:02 scan_results_backup.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 scan_results.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 12:39 secret_file.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 secret.key
drwxrwxr-x 2 kali kali 4096 Dec 12 11:11 temp_files
```

18. Modify permissions using the symbolic method:

```
chmod u+x admin_tool.sh
```

```
(kali@kali)-[~/Lab1]
$ chmod u+x admin_tool.sh

(kali@kali)-[~/Lab1]
$ ls -l
total 36
-rwxrw-r-- 1 kali kali 0 Dec 12 12:39 admin_tool.sh
```

Make script runnable by owner using **chmod u+x admin_tool.sh** these grants execute access via the **x** command. So now the user can launch it directly from terminal

The **ls -l** result changes from **-rw-rw-r--** to **-rwxrw-r--** so the file owner gains execute access

```
chmod o-r secret_file.txt
```

```
(kali@kali)-[~/Lab1]
$ chmod o-r secret_file.txt

(kali@kali)-[~/Lab1]
$ ls -l
-rw-rw- 1 kali kali 0 Dec 12 12:39 secret_file.txt
```

chmod o-r secret_file.txt – takes away read access for everyone else using minus sign.
Check files using **ls -l** before: **-rw-r--r-- secret_file.txt** to **-rw-r----- secret_file.txt**
other people aren't able to see the document anymore they lose access


```
chmod a+w public_read.txt
```

```
(kali㉿kali)-[~/Lab1]
$ chmod a+w public_read.txt

(kali㉿kali)-[~/Lab1]
$ ls -l
-rw-rw-rw- 1 kali kali    0 Dec 12 12:39 public_read.txt
```

chmod gives write access to everyone on public_read.txt using a+w command.
changes from: **-rw-r--r--** public_read.txt to **-rw-rw-rw-** public_read.txt

Now anybody's able to tweak the document. Right now, anyone can make change

19. Modify permissions using the octal method:

```
chmod 755 admin_tool.sh
```

```
(kali㉿kali)-[~/Lab1]
$ chmod 755 admin_tool.sh

(kali㉿kali)-[~/Lab1]
$ ls -l
total 36
-rwxr-xr-x 1 kali kali    0 Dec 12 12:39 admin_tool.sh
```

chmod 755 admin_tool.sh gives user full access group gets read plus run ability others can just view or launch it **ls -l** shows **-rwxr-xr-x** the file runs for all users yet only owner may change its contents.

```
chmod 600 secret_file.txt
```

```
(kali㉿kali)-[~/Lab1]
$ chmod 600 secret_file.txt

(kali㉿kali)-[~/Lab1]
$ ls -l
-rw----- 1 kali kali    0 Dec 12 12:39 secret_file.txt
```

chmod 600 secret_file.txt gives only the user power to view or edit it. **ls -l** shows this now **-rw----- secret_file.txt** Nobody else can access it not group, not others either. The file's locked down tight, Only the owner can see this file.

```
chmod 644 public_read.txt
```

```
(kali㉿kali)-[~/Lab1]
$ chmod 644 public_read.txt

(kali㉿kali)-[~/Lab1]
$ ls -l
total 36
-rwxr-xr-x 1 kali kali 0 Dec 12 12:39 admin_tool.sh
-rw-rw-r-- 1 kali kali 12 Dec 12 11:48 config_new.txt
-rw-rw-r-- 1 kali kali 12 Dec 12 11:45 config_old.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 data.xml
drwxrwxr-x 4 kali kali 4096 Dec 12 09:51 evidence
drwxrwxr-x 2 kali kali 4096 Dec 12 11:06 exploits
-rw-rw-r-- 1 kali kali 79 Dec 12 12:10 logfile.txt
drwxrwxr-x 2 kali kali 4096 Dec 12 11:07 logs
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 passlist.txt
-rw-r--r-- 1 kali kali 0 Dec 12 12:39 public_read.txt
drwxrwxr-x 4 kali kali 4096 Dec 12 09:51 recon
-rw-rw-r-- 1 kali kali 36 Dec 12 11:38 scan_report.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:02 scan_results_backup.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 scan_results.txt
-rw----- 1 kali kali 0 Dec 12 12:39 secret_file.txt
-rw-rw-r-- 1 kali kali 0 Dec 12 11:00 secret.key
drwxrwxr-x 2 kali kali 4096 Dec 12 11:11 temp_files
```

chmod 642 public_read.txt sets access so Owner can read or change it, group sees it, others see it too. Run **ls -l** you get **-rw-r--r-- public_read.txt** to **-rw-r--r- public_read.txt** This means anyone can open it, but only user can edit

4.2: Changing File Ownership

20. Change the ownership of a file:

```
sudo chown root secret_file.txt
ls -l secret_file.txt
```

```
(kali㉿kali)-[~/Lab1]
$ sudo chown root secret_file.txt
[sudo] password for kali:

(kali㉿kali)-[~/Lab1]
$ ls -l secret_file.txt
-rw----- 1 root kali 0 Dec 12 12:39 secret_file.txt
```

The file **secret_file.txt** is now owned by the **root user**, meaning only root has control over it unless permissions are adjusted.

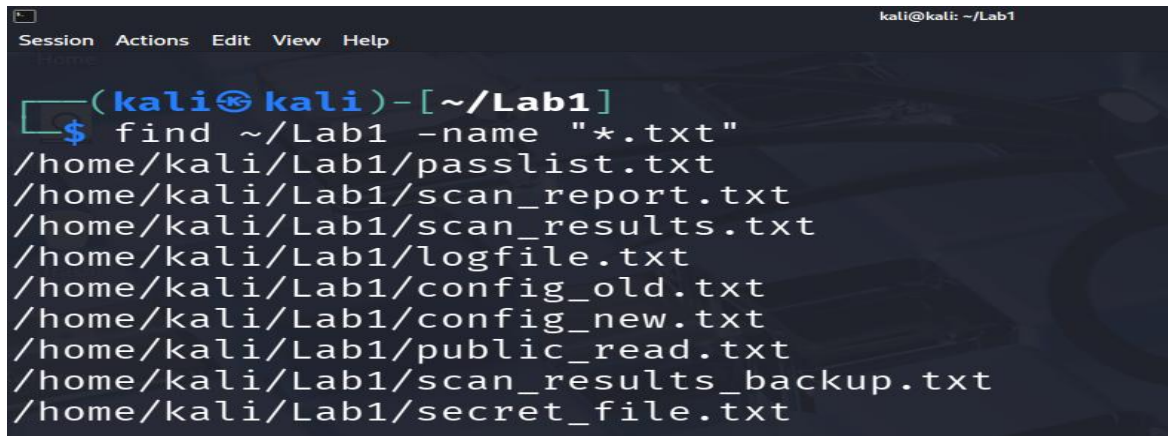
Part 5: Search and Find Operations

Finding files and directories quickly is an essential skill. This section covers the powerful **find** command.

5.1: Finding Files

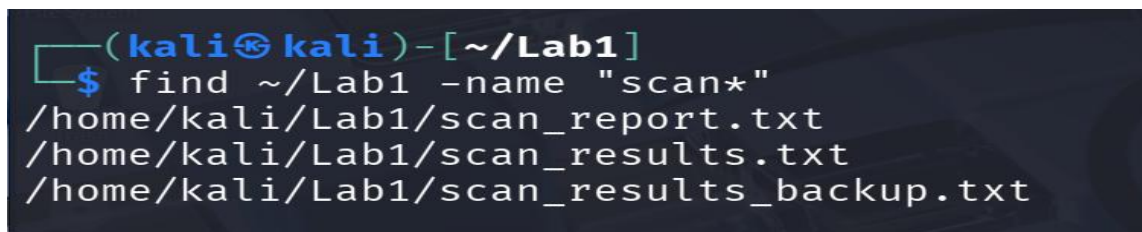
21. Find files by name:

```
find ~/Lab1 -name "*.txt"
```



```
(kali㉿kali)-[~/Lab1]
$ find ~/Lab1 -name "*.txt"
/home/kali/Lab1/passlist.txt
/home/kali/Lab1/scan_report.txt
/home/kali/Lab1/scan_results.txt
/home/kali/Lab1/logfile.txt
/home/kali/Lab1/config_old.txt
/home/kali/Lab1/config_new.txt
/home/kali/Lab1/public_read.txt
/home/kali/Lab1/scan_results_backup.txt
/home/kali/Lab1/secret_file.txt
```

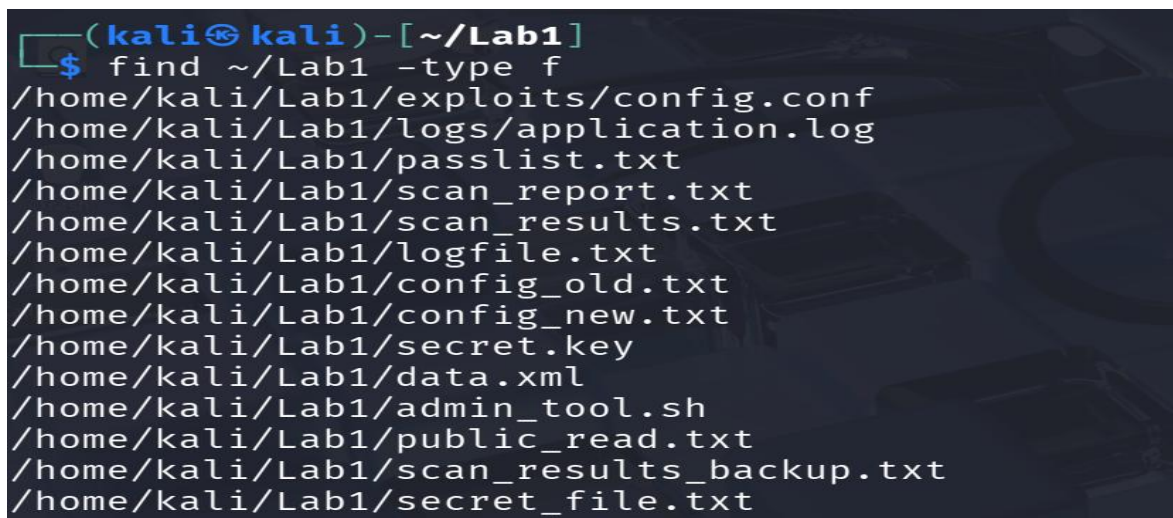
```
find ~/Lab1 -name "scan*"
```



```
(kali㉿kali)-[~/Lab1]
$ find ~/Lab1 -name "scan*"
/home/kali/Lab1/scan_report.txt
/home/kali/Lab1/scan_results.txt
/home/kali/Lab1/scan_results_backup.txt
```

22. Find files by type:

```
find ~/Lab1 -type f
```



```
(kali㉿kali)-[~/Lab1]
$ find ~/Lab1 -type f
/home/kali/Lab1/exploits/config.conf
/home/kali/Lab1/logs/application.log
/home/kali/Lab1/passlist.txt
/home/kali/Lab1/scan_report.txt
/home/kali/Lab1/scan_results.txt
/home/kali/Lab1/logfile.txt
/home/kali/Lab1/config_old.txt
/home/kali/Lab1/config_new.txt
/home/kali/Lab1/secret.key
/home/kali/Lab1/data.xml
/home/kali/Lab1/admin_tool.sh
/home/kali/Lab1/public_read.txt
/home/kali/Lab1/scan_results_backup.txt
/home/kali/Lab1/secret_file.txt
```



```
find ~/Lab1 -type d
```

```
(kali㉿kali)-[~/Lab1]  
$ find ~/Lab1 -type d  
/home/kali/Lab1  
/home/kali/Lab1/exploits  
/home/kali/Lab1/logs  
/home/kali/Lab1/recon  
/home/kali/Lab1/recon/services  
/home/kali/Lab1/recon/hosts  
/home/kali/Lab1/temp_files  
/home/kali/Lab1/evidence  
/home/kali/Lab1/evidence/screenshots  
/home/kali/Lab1/evidence/packets
```

23. Find files by size:

```
find ~/Lab1 -size +1k
```

```
(kali㉿kali)-[~/Lab1]  
$ find ~/Lab1 -size +1k  
/home/kali/Lab1  
/home/kali/Lab1/exploits  
/home/kali/Lab1/logs  
/home/kali/Lab1/recon  
/home/kali/Lab1/recon/services  
/home/kali/Lab1/recon/hosts  
/home/kali/Lab1/temp_files  
/home/kali/Lab1/evidence  
/home/kali/Lab1/evidence/screenshots  
/home/kali/Lab1/evidence/packets
```

```
find ~/Lab1 -size -100c
```

```
(kali㉿kali)-[~/Lab1]  
$ find ~/Lab1 -size -100c  
/home/kali/Lab1/exploits/config.conf  
/home/kali/Lab1/logs/application.log  
/home/kali/Lab1/passlist.txt  
/home/kali/Lab1/scan_report.txt  
/home/kali/Lab1/scan_results.txt  
/home/kali/Lab1/logfile.txt  
/home/kali/Lab1/config_old.txt  
/home/kali/Lab1/config_new.txt  
/home/kali/Lab1/secret.key  
/home/kali/Lab1/data.xml  
/home/kali/Lab1/admin_tool.sh  
/home/kali/Lab1/public_read.txt  
/home/kali/Lab1/scan_results_backup.txt  
/home/kali/Lab1/secret_file.txt
```


Exercise

- What's the difference between the outputs of `find ~/Lab1 -type f` and `find ~/Lab1 -type d`?

Explanation

Searching with `-type F` finds plain files, going through every subdirectory it meets, yet skips directories, showing only items such as “.txt” or “.jpg” stuff

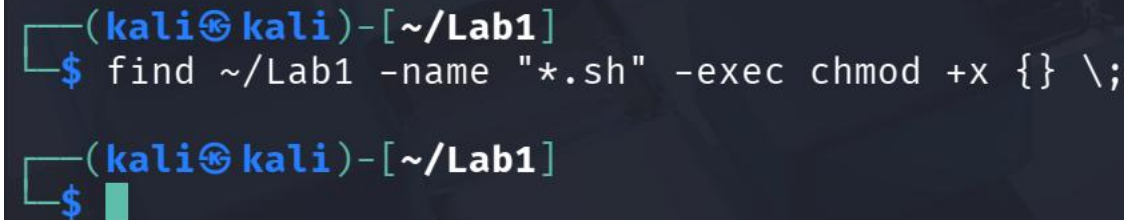
On the flip side, `-type d` shows just folders no matter how deep they are instead of files, it skips right to the folders names things like “recon” or “logs”

In short note: `-type f` spots Files, whereas `-type d` finds folders

5.2: Finding and Executing Commands

24. Find and execute a command on the results:

```
find ~/Lab1 -name "*.sh" -exec chmod +x {} \;
```



A terminal window with a dark background and light blue text. The prompt is `(kali㉿kali)-[~/Lab1]`. The user enters `$ find ~/Lab1 -name "*.sh" -exec chmod +x {} \;`. The prompt changes to `(kali㉿kali)-[~/Lab1]` and a green cursor is visible after the dollar sign.

Exercise

- What did this command accomplish?

does two things:

- Finds every `.sh` file in `~/Lab1`, also checking each folder inside it.
- Makes every script runnable using `chmod +x`.

In simple terms

- name `".sh"` - Find files that end with `.sh`, typically shell scripts.
- Run `chmod +x` on each match to grant run access.

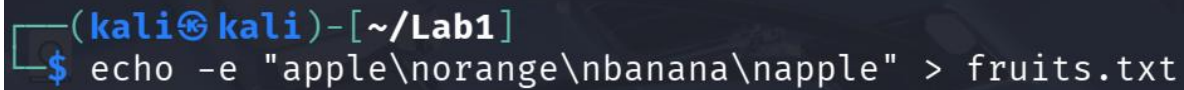
Part 6: Text Processing Challenges

Text processing is a common task in cybersecurity, from analyzing logs to manipulating wordlists. This section introduces some powerful text processing tools

6.1: Sorting and Uniquing

25. Create a file with some sample data:

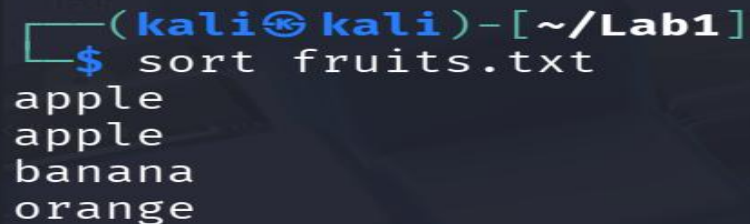
```
echo -e "apple\norange\nbanana\napple" > fruits.txt
```



```
(kali㉿kali)-[~/Lab1]  
$ echo -e "apple\norange\nbanana\napple" > fruits.txt
```

26. Sort the content of the file:

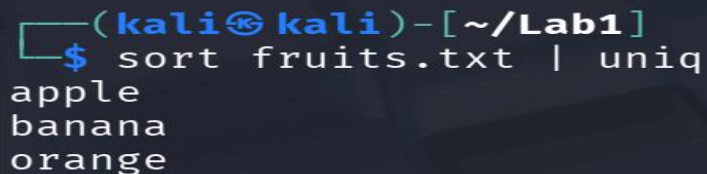
```
sort fruits.txt
```



```
(kali㉿kali)-[~/Lab1]  
$ sort fruits.txt  
apple  
apple  
banana  
orange
```

27. Sort and remove duplicate lines:

```
sort fruits.txt | uniq
```

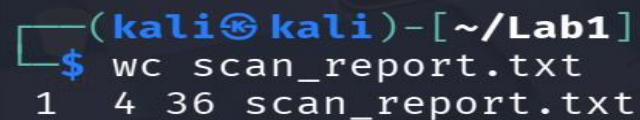


```
(kali㉿kali)-[~/Lab1]  
$ sort fruits.txt | uniq  
apple  
banana  
orange
```

6.2: Word Counting

28. Count the lines, words, and characters in a file:

```
wc scan_report.txt
```



```
(kali㉿kali)-[~/Lab1]  
$ wc scan_report.txt  
1  4 36 scan_report.txt
```

29. Count only the lines in a file:

```
wc -l logfile.txt
```



```
(kali㉿kali)-[~/Lab1]  
$ wc -l logfile.txt  
3 logfile.txt
```

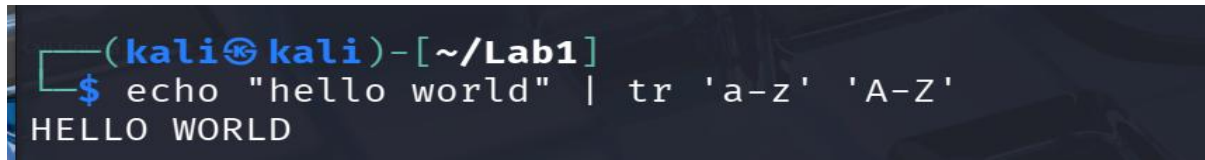
Exercise

- What do the three numbers from wc represent?

6.3: Text Transformation

30. Transform text from lowercase to uppercase:

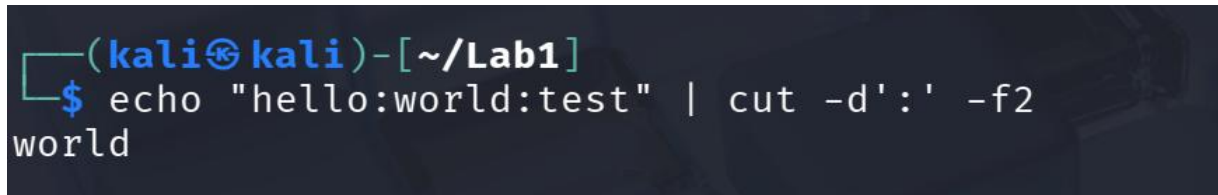
```
echo "hello world" | tr 'a-z' 'A-Z'
```



```
(kali㉿kali)-[~/Lab1]
$ echo "hello world" | tr 'a-z' 'A-Z'
HELLO WORLD
```

31. Extract specific fields from a line of text:

```
echo "hello:world:test" | cut -d':' -f2
```



```
(kali㉿kali)-[~/Lab1]
$ echo "hello:world:test" | cut -d':' -f2
world
```

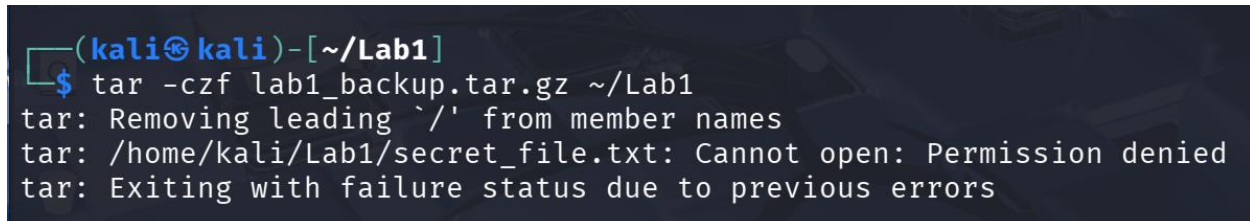
Part 7: Compression and Archiving

Archiving and compressing files are essential for storage and transfer. This section covers the tar command.

7.1: Creating and Extracting Archives

32. Create a compressed archive of your Lab1 directory:

```
tar -czf lab1_backup.tar.gz ~/Lab1
```



```
(kali㉿kali)-[~/Lab1]
$ tar -czf lab1_backup.tar.gz ~/Lab1
tar: Removing leading `/' from member names
tar: /home/kali/Lab1/secret_file.txt: Cannot open: Permission denied
tar: Exiting with failure status due to previous errors
```

33. List the contents of the archive:

```
tar -tzf lab1_backup.tar.gz
```



```

(kali@kali)-[~/Lab1]
$ tar -tzf lab1_backup.tar.gz
home/kali/Lab1/
home/kali/Lab1/exploits/
home/kali/Lab1/exploits/config.conf
home/kali/Lab1/fruits.txt
home/kali/Lab1/logs/
home/kali/Lab1/logs/application.log
home/kali/Lab1/passlist.txt
home/kali/Lab1/recon/
home/kali/Lab1/recon/services/
home/kali/Lab1/recon/hosts/
home/kali/Lab1/scan_report.txt
home/kali/Lab1/scan_results.txt
home/kali/Lab1/logfile.txt
home/kali/Lab1/config_old.txt
home/kali/Lab1/config_new.txt
home/kali/Lab1/secret.key
home/kali/Lab1/data.xml
home/kali/Lab1/admin_tool.sh
home/kali/Lab1/public_read.txt
home/kali/Lab1/lab1_backup.tar.gz
home/kali/Lab1/scan_results_backup.txt
home/kali/Lab1/temp_files/
home/kali/Lab1/evidence/
home/kali/Lab1/evidence/screenshots/
home/kali/Lab1/evidence/packets/

```

34. Extract the archive to a new directory:

```

mkdir extracted
tar -xzf lab1_backup.tar.gz -C extracted/

```

Exercise

- Verify the extraction was successful.

```

(kali@kali)-[~/Lab1]
$ tar -xzf lab1_backup.tar.gz -C extracted/

(kali@kali)-[~/Lab1]
$ cd extracted

(kali@kali)-[~/Lab1/extracted]
$ ls
home

(kali@kali)-[~/Lab1/extracted]
$ cd home

(kali@kali)-[~/Lab1/extracted/home]
$ ls
kali

(kali@kali)-[~/Lab1/extracted/home]
$ cd kali

(kali@kali)-[~/Lab1/extracted/home/kali]
$ ls
Lab1

(kali@kali)-[~/Lab1/extracted/home/kali]
$ cd Lab1

(kali@kali)-[~/Lab1/extracted/home/kali/Lab1]
$ ls
admin_tool.sh  fruits.txt      recon
config_new.txt lab1_backup.tar.gz scan_report.txt
config_old.txt logfile.txt      scan_results_backup.txt
data.xml       logs            scan_results.txt
evidence       passlist.txt    secret.key
exploits       public_read.txt temp_files

```

All the content is visible and accessible — this confirms the **tar.gz** archive was extracted successfully and the directory structure is intact.

Part 8: System Monitoring Exercises

Monitoring system resources is a crucial aspect of system administration and security.

8.1: Process and Disk Usage Monitoring

35. Monitor running processes:

```
ps aux
```

```
(kali㉿kali)-[~/Lab1]
$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 24876 10980 ?        Ss   06:15   0:18 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    06:15   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    06:15   0:00 [pool_workqueue_rel
root           4  0.0  0.0      0     0 ?        I<   06:15   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   06:15   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   06:15   0:00 [kworker/R-kvfree_r
root           7  0.0  0.0      0     0 ?        I<   06:15   0:00 [kworker/R-slub_flu
root           8  0.0  0.0      0     0 ?        I<   06:15   0:00 [kworker/R-netns]
root          12  0.0  0.0      0     0 ?        I    06:15   0:00 [kworker/u512:0-ipv
root          13  0.0  0.0      0     0 ?        I<   06:15   0:00 [kworker/R-mm_percp
root          14  0.1  0.0      0     0 ?        S    06:15   0:59 [ksoftirqd/0]
root          15  0.5  0.0      0     0 ?        I    06:15   3:07 [rcu_preempt]
root          16  0.0  0.0      0     0 ?        S    06:15   0:00 [rcu_exp_par_gp_kth
root          17  0.0  0.0      0     0 ?        S    06:15   0:00 [rcu_exp_gp_kthread
root          18  0.0  0.0      0     0 ?        S    06:15   0:10 [migration/0]
root          19  0.0  0.0      0     0 ?        S    06:15   0:00 [idle_inject/0]
root          20  0.0  0.0      0     0 ?        S    06:15   0:00 [cpuhp/0]
root          21  0.0  0.0      0     0 ?        S    06:15   0:00 [cpuhp/1]
root          22  0.0  0.0      0     0 ?        S    06:15   0:00 [idle_inject/1]
```

```
ps -ef | grep kali
```

```
(kali㉿kali)-[~/Lab1]
$ ps -ef | grep kali
kali          3179  0 06:25 ?        00:00:02 /usr/lib/systemd/systemd --user
kali          3196  3179 0 06:25 ?        00:00:00 (sd-pam)
kali          3219  3179 0 06:25 ?        00:00:05 /usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
kali          3221  3179 0 06:25 ?        00:00:00 /usr/bin/pipewire
kali          3222  3179 0 06:25 ?        00:00:01 /usr/bin/gnome-keyring-daemon --foreground --components=pkcs11,secrets --control-directory=/run/user/1002/keyrin
g
kali          3223  3179 0 06:25 ?        00:00:00 /usr/bin/mpiris-proxy
kali          3224  3179 0 06:25 ?        00:00:01 /usr/bin/wireplumber
kali          3225  3179 0 06:25 ?        00:00:00 /usr/bin/pipewire -c filter-chain.conf
kali          3226  3179 0 06:25 ?        00:00:00 /usr/bin/pipewire-pulse
kali          3235  3167 0 06:25 ?        00:00:15 xfce4-session
kali          3319  3179 0 06:25 ?        00:00:00 /usr/libexec/at-spi-bus-launcher
kali          3326  3319 0 06:25 ?        00:00:02 /usr/bin/dbus-daemon --config-file=/usr/share/defaults/at-spi2/accessibility.conf --nofork --print-address 12 --
address=unix:path=/run/user/1002/at-spi/bus_0
kali          3336  3179 0 06:25 ?        00:00:08 /usr/libexec/at-spi2-registryd --use-gnome-session
kali          3344  1 0 06:25 ?        00:00:00 /usr/bin/ssh-agent -s
kali          3353  3179 0 06:25 ?        00:00:05 /usr/bin/gpg-agent --supervised
kali          3356  3235 2 06:25 ?        00:18:00 xfwm4
kali          3361  3179 0 06:25 ?        00:00:00 /usr/libexec/gvfsd
kali          3367  3179 0 06:25 ?        00:00:00 /usr/libexec/gvfsd-fuse /run/user/1002/gvfs -f
kali          3377  3235 0 06:25 ?        00:00:28 xfsettingsd
kali          3383  3179 0 06:25 ?        00:00:00 /usr/libexec/dconf-service
kali          3387  3235 0 06:25 ?        00:00:40 xfce4-panel
kali          3393  3235 0 06:25 ?        00:00:03 Thunar --daemon
```

36. Check disk usage:

```
df -h
```

```
(kali㉿kali)-[~/Lab1]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            1.8G     0   1.8G   0% /dev
tmpfs           389M   1.4M  387M   1% /run
/dev/sda1       76G    43G   29G  61% /
tmpfs           1.9G   4.0K   1.9G   1% /dev/shm
none            1.0M     0   1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.9G  592K   1.9G   1% /tmp
none            1.0M     0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs           389M  104K  389M   1% /run/user/1000
tmpfs           389M  112K  389M   1% /run/user/1002
```

```
du -sh ~/Lab1
```

```
(kali@kali)-[~/Lab1]
$ du -sh ~/Lab1
136K    /home/kali/Lab1
```

Exercise

- How much space does your Lab1 directory use?

Answer:

The Total size of everything inside Lab1 directory {files + Folders} is **136k**

8.2: System Information

37. Get system information:

```
uname -a
```

```
(kali@kali)-[~/Lab1]
$ uname -a
Linux kali 6.16.8+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24) x86_64 GNU/Linux
```

System information summary

Operating System (OS): Linux (Kali Linux)

Hostname: kali

Kernel version: 6.16.8+kali-amd64

Kernel build: #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1

Architecture: x86_64 (64-bit)

Build date: September 24, 2025

You are running **Kali Linux 64-bit with kernel 6.16.8.**

```
whoami
```

```
(kali@kali)-[~/Lab1]
$ whoami
kali
```

This means **your current logged-in user is: kali.**

Challenge Exercises

Challenge 1: Directory Structure for Penetration Test

Create a directory structure for a penetration test engagement with these requirements:

- Main directory called "Engagement_2024"
- Subdirectories for: reconnaissance, exploitation, reporting, evidence
- In the evidence directory, create subfolders for: screenshots, logs, packets
- Create a readme.txt file in each main directory

Step 1: Create the directory structure for Penetration Test

```
(kali㉿kali)-[~]  
$ mkdir -p Engagement_2024/{reconnaissance,exploitation,reporting,evidence/{screenshots,logs,packets}}  
  
(kali㉿kali)-[~]  
$
```

These builds:

- Engagement_2024 as the main directory
- Subdirectories: reconnaissance, exploitation, reporting, evidence
- Evidence subfolders: screenshots, logs, packet

Step 2: Create readme.txt in each main subdirectory

```
(kali㉿kali)-[~]  
$ cd Engagement_2024  
  
(kali㉿kali)-[~/Engagement_2024]  
$ touch {reconnaissance,exploitation,reporting,evidence}/readme.txt
```

This places a **readme.txt** file inside each of the four main subdirectories.

Final Directory Structure for Penetration Test

```
(kali㉿kali)-[~/Engagement_2024]  
$ cd ../  
  
(kali㉿kali)-[~]  
$ tree Engagement_2024  
Engagement_2024  
├── evidence  
│   ├── logs  
│   ├── packets  
│   ├── readme.txt  
│   └── screenshots  
├── exploitation  
│   └── readme.txt  
├── reconnaissance  
│   └── readme.txt  
└── reporting  
    └── readme.txt  
  
8 directories, 4 files
```

The Final Directory structure we are able to create 8 directories and 4 files for our penetration testing

Challenge 2: Permission Scheme Setup

Set up the following permission scheme:

- All .sh files should be executable only by the owner.

```
(kali㉿kali)-[~]  
$ find ~/Engagement_2024 -name "*.sh" -exec chmod 700 {} \;  
  
(kali㉿kali)-[~]  
$
```

Ran the command `find ~/Engagement_2024 -name ".sh" -exec chmod 700 {} \;` saw nothing pop up since no .sh files exist in **Engagement_2024** right now. So first off, gotta make one of those .sh files - only then can it turn in to something runnable just by its owner

```
cd ~/Engagement_2024  
touch penetration_lab.sh  
find ~/Engagement_2024 -name "*.sh" -exec chmod 700 {} \;  
ls -l penetration_lab.sh
```

```
(kali㉿kali)-[~]  
$ cd ~/Engagement_2024  
  
(kali㉿kali)-[~/Engagement_2024]  
$ touch penetration_lab.sh  
  
(kali㉿kali)-[~/Engagement_2024]  
$ find ~/Engagement_2024 -name "*.sh" -exec chmod 700 {} \;  
  
(kali㉿kali)-[~/Engagement_2024]  
$ ls -l penetration_lab.sh  
-rwx----- 1 kali kali 0 Dec 13 05:25 penetration_lab.sh
```

- All .txt files should be readable by everyone but only writable by the owner.

```
Cd Engagement_2024  
find ~/Engagement_2024 -name "*.txt" -exec chmod 644 {} \;  
find ~/Engagement_2024 -name "*.txt" -exec ls -l {} \;
```

```
(kali㉿kali)-[~/Engagement_2024]  
$ find ~/Engagement_2024 -name "*.txt" -exec chmod 644 {} \;
```

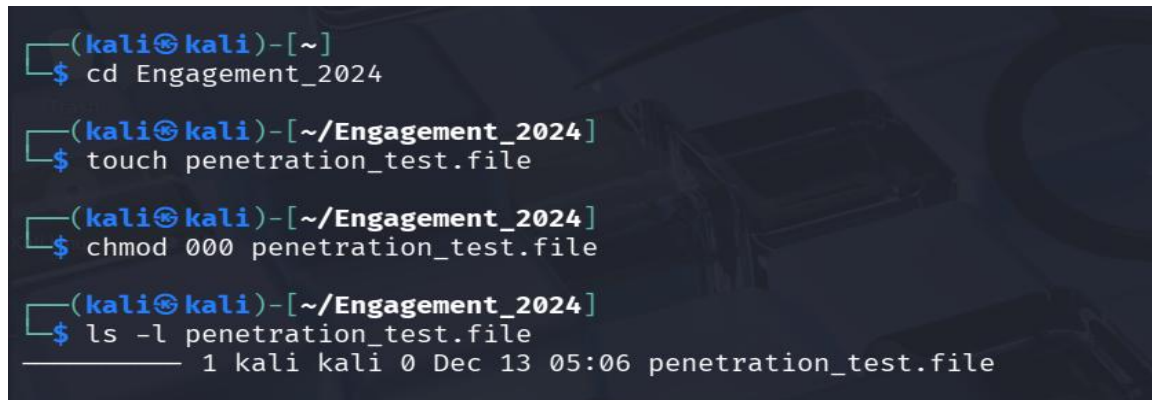
```
(kali㉿kali)-[~]  
$ find ~/Engagement_2024 -name "*.txt" -exec ls -l {} \;  
-rw-r--r-- 1 kali kali 0 Dec 13 04:35 /home/kali/Engagement_2024/reconnaissance/readme.txt  
-rw-r--r-- 1 kali kali 0 Dec 13 04:35 /home/kali/Engagement_2024/exploitation/readme.txt  
-rw-r--r-- 1 kali kali 0 Dec 13 04:35 /home/kali/Engagement_2024/reporting/readme.txt  
-rw-r--r-- 1 kali kali 0 Dec 13 04:35 /home/kali/Engagement_2024/evidence/readme.txt
```

- The `'644'` octal number (`'-rw-r--r--'`) defines basic file access:
- The owner can both read and write (view and edit).
- Group members can only read (view).
- Everyone else also gets read-only access (can view, but not change anything)

Most files are set up like this everyone's allowed to view, yet just the creator gets to edit..

- Create one file that nobody can read, write, or execute.

```
Cd Engagement_2024
Touch penetration_test.file
Chmod 000 penetration_test.file
Ls -l penetration_test.file
```



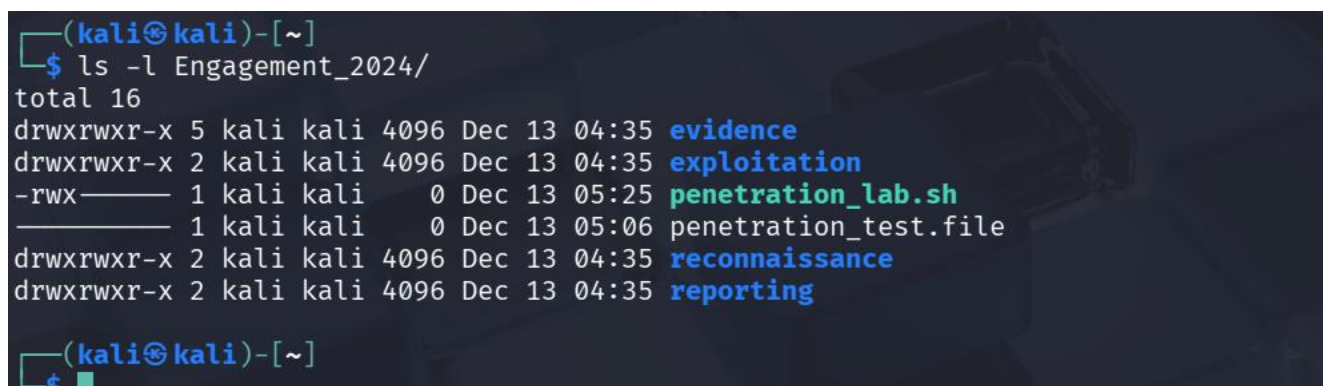
```
(kali㉿kali)-[~]
$ cd Engagement_2024

(kali㉿kali)-[~/Engagement_2024]
$ touch penetration_test.file

(kali㉿kali)-[~/Engagement_2024]
$ chmod 000 penetration_test.file

(kali㉿kali)-[~/Engagement_2024]
$ ls -l penetration_test.file
----- 1 kali kali 0 Dec 13 05:06 penetration_test.file
```

- We ensured our custom shell scripts ran just for the person who owned them. That way, no one else could use them - unless they were allowed.
- Our test files were set up tight on purpose - no one could get into them, period. That way, it'd be obvious we'd actually stopped access.
- The main project folders **Engagemet_2024** got organized to help folks move around easier. Still, despite this setup, access stayed limited based on who you were and what tasks mattered to you.



```
(kali㉿kali)-[~]
$ ls -l Engagement_2024/
total 16
drwxrwxr-x 5 kali kali 4096 Dec 13 04:35 evidence
drwxrwxr-x 2 kali kali 4096 Dec 13 04:35 exploitation
-rwx----- 1 kali kali 0 Dec 13 05:25 penetration_lab.sh
----- 1 kali kali 0 Dec 13 05:06 penetration_test.file
drwxrwxr-x 2 kali kali 4096 Dec 13 04:35 reconnaissance
drwxrwxr-x 2 kali kali 4096 Dec 13 04:35 reporting

(kali㉿kali)-[~]
$
```

Challenge 3: Log Analysis Using Command Line

Using only command-line tools:

- Find all .log files in your home directory.
- Count how many lines contain the word "ERROR".
- Save the results to a file called error_report.txt.

Conclusion

You have now practiced comprehensive Linux command-line operations that form the foundation of efficient security work in Kali Linux. These skills in file management, permission handling, text processing, and system navigation are essential for any cybersecurity professional.

Please continue to practice these commands and explore their various options to become a true master of the command line. The proficiency you develop in this lab will serve you well throughout your cybersecurity career.

Additional Resources

- Kali Linux Documentation: <https://www.kali.org/docs/>
- Linux Command Library: <https://linuxcommand.org/>
- GNU Coreutils Manual: <https://www.gnu.org/software/coreutils/manual/coreutils.html>